

WAVE 2工作

Code review问题

startup/cleanup问题对齐

因为在本项目中，停的是同步 Python 函数，跑在 `anyio.to_thread.run_sync` 开的普通线程里。取消只能作用在「谁在等这个线程」，不能作用在「线程里正在执行的字节码」。

CPython 没有安全的 `Thread.kill()`。线程要停，只能自己走到 `return`，或者进程一起死。

Startup 失败后，FastAPI lifespan 会 `await shutdown_runtime()`。

async 等待 → `anyio.to_thread.run_sync` → 同步 `stop_all()` / `scheduler.shutdown()` / `thread.join()`

取消只能作用在左边的 `await`。右边线程里的字节码、`join()`、`Docker HTTP`，CPython 没有安全的 `Thread.kill()`。线程要停，只能自己 `return`，或者整个进程一起死。

一旦 `stop_all()` 进了某个同步 `stop`：

1. 遗弃 worker (旧代码： `fail_after` + `abandon_on_cancel=True`)
lifespan 可以在 deadline 内返回，但线程还在跑，握着 `TaskRegistry` 锁，继续改已经挂到 `app.state.runtime_handle` 上的对象。这就是原来的 P1。
2. 等它跑完 (现在的选择)
不会有悬挂线程，也不会在申报失败之后还偷偷改 `handle`。代价是：这次 `await shutdown_runtime()` 要等到 `stop` 自己结束。

所以「超时取消 cleanup」在本系统的架构里只有两种真实语义：

1. 继续等 (worker 自己结束)
2. 不等了，但 worker 还活着 (遗弃)

后续任务

Wave 2 的范围：

- G2: M04 内置 Agent 迁移
- G3: M06 内置工具迁移，但暂不接 trading tool

- G4: M08 内置 Prompt 迁移、M13 基础 Catalog 装载
- G5: M20 行情与缓存边界
- G7: M12 数据库 schema 与配置迁移
- G10: M17 SDK/示例骨架
- G1、G6、G8 在 Wave 2 没有独立主模块
- G9 要到 Wave 4，但目前没有明确负责人，需要后续补充

人员任务

人员	Wave 2 主任务	主要交付
傅雷鸣	M04 集成、M06 非交易工具； ReAct、Tool Agent迁移	Agent 注册集成、内置工具拆分、 兼容清理
温彦博	M08、M13 基础部分；Rule Agent迁移	Prompt 文件化、扩展发现与 Catalog
王一鼎	M12 migration/schema； memory Agent迁移	账户 runtime config 表、迁移、 repository
舒杨	多智能体迁移	多智能体Agent迁移
刘轩同	M20；维护 M11 接口	Market facade、三类 cache 边 界；为 M06 提供 Gateway
卢屿阳	M17 示例骨架	SDK testing API、扩展示例、教程 初稿

其他的部分照着文档做就行了比较简单

内置 Agent 迁移指南（以 ReAct 为模板）

本文给后续迁移 `core.advanced-multi-agent`、`core.rule-aware` 的人用。方法以已经落地的 `core.react` 为准，不要另起一套包装方式。

在分支 `refactor/main` 上已经推送了 ReAct Agent 的迁移示例，可以直接让 coding agent 参考着写

1. 要达成什么

把现有 Agent 注册成公开组件，让系统和第三方走同一套 SPI：

```
1 AgentFactory.create(AgentBuildContext, config) -> Agent
2 Agent.run(DecisionContext) -> AgentRunResult
```

行为必须保持：Prompt 文本、步骤顺序、工具调用顺序、终止原因、交易副作用都与迁移前等价。算法留在原文件，本任务只加 adapter，不重写决策循环。

2. 明确不要做什么

不要	原因
把旧 <code>run()</code> 改成 <code>async</code>	系统按账户 worker 同步调用；返回 <code>coroutine</code> 会被 <code>Runtime</code> 拒绝
重写 Prompt 文案或决策算法	属于行为变化，不在 M04
改 <code>ai_decision_service.py</code> / 生产调度	归 M10
把 step 写入 <code>AgentTrace</code> 表	归 M16；本波只 emit 公开事件
把旧 Agent 改成只依赖公开 <code>ToolInvoker</code>	归 M06；本波仍注入现有 <code>ToolRegistry</code>
为 <code>react</code> 和 <code>react_tool</code> 注册两个 component	它们是同一个 <code>core.react</code> ，用 config 区分

生产路径目前仍走 `create_agent()` shim，返回内部旧 Agent。M04 允许这个过渡层存在；不要提前把生产切到 `AgentRuntime`。

3. 调用链

3.1 目标形态（Runtime 路径，测试和后续 M10 使用）

代码块

```

1  账户 worker 线程
2      -> AgentRuntime.run(AgentSelection, DecisionContext)
3      -> Registry.validate_config()          # schema 默认值写入副本
4      -> Factory.create(AgentBuildContext, config)
5      -> Adapter.run(DecisionContext)
6          -> 把 DecisionContext 转成旧 portfolio / prices
7          -> 现有 XxxAgent.run(...)          # 算法未改
8          -> 把 dict 结果转成 AgentRunResult
9          -> on_step 同步 emit agent.step

```

AgentRuntime 自己会发 `agent.started` / `agent.completed` / `agent.failed` / `agent.cancelled`。Adapter 只负责逐步回调对应的 `agent.step`。

3.2 当前生产形态

代码块

```

1  ai_decision_service
2      -> create_agent(agent_type, llm, tools, **kwargs)
3      -> Registry 按旧名映射到 core.*
4      -> Factory.create(...)
5      -> 若得到 Adapter, 取出 .legacy_agent 返回
6      -> 旧 Agent.run(portfolio, prices, on_step=写库回调)

```

因此 Adapter 必须暴露 `legacy_agent`，类型是原来的 `BaseAgent` 子类。shim 用 `NullEventSink()`，生产 trace 仍走旧 `on_step`，互不干扰。

3.3 为什么要写Adaptor

旧 `ReActAgent.run()` 吃的是 `portfolio` / `prices` 这种 dict，吐的也是 dict。新系统要求的是：

`Agent.run(DecisionContext) -> AgentRunResult`

Adapter 不参与思考、不下单、不改循环。每次运行它只做三步：

1. 把新的 `DecisionContext` 转成旧 Agent 认识的 `portfolio` / `prices`
2. 原样调用现有 `ReActAgent.run(...)`
3. 把返回的 dict 转成 `AgentRunResult`，并把原来的 `on_step` 转成 `agent.step` 事件

所以算法还在旧类里。Adapter 让 Runtime 可以按公开 SPI 跑它，同时 `create_agent()` 仍能拿出里面的 `legacy_agent` 给现在的生产路径用。

4. 文件放哪里

以 ReAct 为准：

代码块

```
1 backend/benchmark/builtin/agents/
2   __init__.py                # register_builtin_agents(), 所有内置 Factory 在此
                                注册
3   _legacy_context.py        # 共用 DecisionContext <-> 旧 dict; 后来者直接复用
4   react.py                  # Descriptor、schema、Factory、Adapter
5   <your_agent>.py           # 新增，不要塞进 services/agent/factory.py
6
7 backend/services/agent/<existing>.py
8                               # 旧类留在原文件；只允许为接入 SPI 做最小接线
9                               # （例如显式 termination_reason、可注入
                                prompt_resolver)
10
11 backend/tests/agents/builtin/
12   test_<your_agent>_migration.py
```

`register_builtin_agents()` 是唯一注册入口。`factory.py` 里的 `_LegacyAgentFactory` + `_build_*` 是还没迁完的三类 Agent 的临时壳；你迁完自己那一类后，要从这个 tuple 里删掉对应条目，否则 `(id, version)` 会 `ComponentConflictError`。

5. 迁移步骤（照 ReAct 做）

步骤 1：定 component id 和一份 config schema

--	--	--

旧 agent_type	component id	现有实现
react / default	core.react	ReActAgent
multi_agent	core.multi-agent	MultiAgent
advanced_multi_agent	core.advanced-multi-agent	AdvancedMultiAgent
rule_aware	core.rule-aware	RuleAwareAgent

id 与版本写死在 Descriptor 上。schema 必须 `additionalProperties: false`，默认值必须等于当前 `AgentConfig` / 构造函数默认值，这样空 config 跑出来的行为和现在一致。

ReAct 的关键点：

- `react` 与 `react_tool` 不是两个 component。评测标签是分析脚本推出来的；运行时只看 `config.tool_routing_enabled`。
- `memory_enabled` 默认是 `null`，不是 `false`。省略时按 registry 是否注册了 `memory_add` / `memory_search` 探测；显式 `false` 必须仍能关掉 Memory Prompt。不要把「没写这个字段」实现成静默关闭。

步骤 2：写 Factory，只从 `AgentBuildContext` 取依赖

代码块

```
1 class XxxAgentFactory:
```

```
def create(self, context: AgentBuildContext, config: Mapping[str, Any]) -> XxxAgentAdapter:
```

...

代码块

```
1
2 Factory 里要做的事：
3
4 1. 校验 `context` 类型。
5 2. 当前内置 Agent 仍需要 `services.agent.tools.ToolRegistry`。若调用方只给了公开 `ToolInvoker`，抛 `ComponentConfigError`（ReAct 的 code 是
```

‘REACT_LEGACY_TOOL_REGISTRY_REQUIRED’)。不要在本波发明第二套工具适配。

3. LLM 可能是 ‘LegacyLLMClientAdapter’。用 ‘legacy_client’ 解包出旧 ‘LLMClient’；解包失败就报配置错误，不要猜。

4. 用 schema 注入后的 ‘config’ 构造旧 Agent。需要的键直接 ‘config["..."]’ 读取——Registry 已经把 default 写进副本了。

5. Prompt 走 ‘get_prompt_resolver(context.prompts)’，不要再 import Prompt 常量模块。若旧类还在 ‘.format()’ 常量，允许在旧文件里加 ‘prompt_resolver’ 参数，但渲染结果必须与原来逐字相同。

6. 把 ‘context.events’ 交给 Adapter，供 ‘agent.step’ 使用。

步骤 3：写 Adapter，同步 ‘run(DecisionContext)’

Adapter 是公开 ‘Agent’。它不做决策，只做三件事：转输入、调旧 ‘run()’、转输出。

输入转换复用 ‘benchmark.builtin.agents._legacy_context’：

– ‘portfolio_from_context()’ / ‘prices_from_context()’ 的 dict 形状必须对齐生产里 ‘ai_decision_service._get_portfolio_data’ 交给 Agent 的对象。数量为 0 的持仓不要塞进 positions。

– 把 ‘context.trace_id’、‘context.decision_round_id’ 原样传给旧 ‘run()’。

输出转换：

– 旧 ‘run()’ 必须返回 ‘dict’，且带合法 ‘termination_reason’ 字符串。缺了或值不在 ‘TerminationReason’ 里，抛 ‘AgentRuntimeError’，不要默认为 HOLD。

– ‘executed_trades’ 用 ‘executed_trades_from_legacy()’。缺 symbol/market 的 payload **不准编造**；放进 ‘metadata["incomplete_executed_trades"]’。带 ‘error’ 的放进 ‘metadata["trade_errors"]’。

– ‘hold’ 以及空的 ‘close_all’（没有实际平掉的仓）不是成交。ReAct 用 ‘_has_filled_trade()’ 区分「调了 execute_trade」和「真的成交」；‘termination_reason’ 不能只因为出现了 ‘<TRADE_DONE>’ 就标 ‘trade_done’。

– ‘AgentRunResult.trace_id’ / ‘decision_round_id’ 必须等于输入 context。Runtime 会检查，对不上就失败。

– ‘summary’ 用旧结果的 ‘reason’（或该类 Agent 等价字段），不要另写文案。

同步约束：‘Adapter.run’ 和旧 ‘run’ 都必须是普通函数。用线程 id 测试确认跑在调用方线程。MultiAgent / Advanced 内部如果已经有自己的并发，保持原样，不要改成由 Runtime 调度。

步骤 4：把 ‘on_step’ 转成 ‘agent.step’

公开事件类型白名单是：‘agent.started’、‘agent.completed’、‘agent.failed’、‘agent.cancelled’、‘agent.step’。

Adapter 在旧 ‘on_step’ 里：

1. 仍保存逐步消息（便于测试对照）。

2. ‘events.emit(AgentRuntimeEvent(type="agent.step", ...))’。

```
38 3. `metadata` 必须含 1-based `step_number` 和 `role`; `content` / `tool_calls`  
    / `name` / `tool_call_id` 有则原样带上, 先走 `to_jsonable()`。  
39 4. `role` 缺失或不可序列化字段直接 `AgentRuntimeError`, 不要空字符串或丢掉该步。  
40 5. `occurred_at` 必须是 timezone-aware UTC。  
41 6. **不要写数据库。**  
42  
43 事件顺序应为:  
44  
45 ```text  
46 agent.started  
47 agent.step    # 1..N, 与旧 on_step 次数、顺序一致  
48 agent.completed
```

步骤 5: 注册, 并改 shim 的返回值

在 `register_builtin_agents()` 里 `registry.register(DESCRIPTOR, Factory())`。

`factory.py` 中:

- `_LEGACY_AGENT_IDS` 已有旧名映射, 不要再加 if/elif 选类。
- 迁完后删除 `_create_legacy_registry()` 里对应的 `_LegacyAgentFactory` 注册。
- `create_agent()` 已经支持「Factory 返回 Adapter 则取出 `legacy_agent`」。新 Adapter 按 ReAct 同样暴露该属性即可。
- ReAct 会过滤不属于 schema 的 kwargs (例如 `account_id`)。你的 schema 同样 `additionalProperties: false` 时, shim 也要丢掉生产仍会传入、但新 schema 不收的键, 否则 `create_agent` 会校验失败。

步骤 6: 旧类只做接入所需的最小接线

ReAct 对 `services/agent/react.py` 只动了接入点, 例如:

- 构造函数可注入 `prompt_resolver`、`memory_enabled`、`tool_routing_enabled`;
- 结果 dict 显式写出 `termination_reason`。

不要把循环、工具 guardrail、交易调用搬到 `benchmark/` 里。旧文件继续拥有算法。

